OS ORANGE PROBLEM (UNIT-4)

Name: Ritesh Kumar Sharma

SRN: PES2UG24CS404

SEM: 4

SECTION: G

Github Link:

https://github.com/ritesh-shar/PES2UG24CS404-pes-vcs

Phase 1:

1A:

```
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
Integrity check failed for object!
PASS: integrity check

All Phase 1 tests passed.
```

1B:

```
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2:

2A:

```
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B:

```
332ac201b2d3 | head -n 20
00000000: 7472 6565 2031 3339 0031 3030 3634 3420  tree 139.100644
00000010: 5245 4144 4d45 2e6d 6400 aaaa aaaa aaaa  README.md.......
00000020: aaaa aaaa aaaa aaaa aaaa aaaa aaaa aaaa  ................
00000030: aaaa aaaa aaaa aaaa aaaa 3130 3037 3535  ..........100755
00000040: 2062 7569 6c64 2e73 6800 cccc cccc cccc   build.sh.......
00000050: cccc cccc cccc cccc cccc cccc cccc cccc  ................
00000060: cccc cccc cccc cccc cccc 3430 3030 3020  ..........40000
00000070: 7372 6300 bbbb bbbb bbbb bbbb bbbb bbbb  src.............
00000080: bbbb bbbb bbbb bbbb bbbb bbbb bbbb bbbb  ................
00000090: bbbb bbbb                                ....
```

Phase 3:

3A:

```
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ ./pes add file1.txt file2.txt
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  pes.h
  untracked:  tree.c
  untracked:  pes.c
  untracked:  README.md
  untracked:  Makefile
  untracked:  test_tree.c
  untracked:  object.c
  untracked:  tree.h
  untracked:  index.c
  untracked:  test_objects
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  index.h
```

3B:

```
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776755670 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776755675 6 file2.txt
```

Phase 4:

4 A:

```
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ ./pes log
commit fce3294e2c12d63a4b971d0aed5179c4ad9d28bbae27f6367a7740e51db1a071
Author: Ritesh <PES2UG24CS404>
Date:   1776757803

    Add farewell

commit 8b6684cc26eeb6808a68298be8b1bc7248b618ca12a0434fa824bb6efb53c7d2
Author: Ritesh <PES2UG24CS404>
Date:   1776757790

    Add world

commit d26c424b937840baefb0d18b98a75658c2848fa5cb7b088e33e1ae52ba8c1921
Author: Ritesh <PES2UG24CS404>
Date:   1776757764

    Initial committial commit
```

4 B:

```
.pes/HEAD
.pes/index
.pes/objects/0b/a0c276dc2221ef87ae3b6e977b86df735ac944bdb4bd1df52842153d53f341
.pes/objects/13/b7e821533d3fe3728a3c4560606a65aab99f4390b9df0714f9075c0ef4c2d6
.pes/objects/70/c3c320ec92e88f7e0124321c90f1f49d50da497e7be3761d2e24a776e61970
.pes/objects/77/409d20b3d1bdfce458c50e814d5ee0d9efff0cdf5025f5b13cd7ea124f4346
.pes/objects/8b/6684cc26eeb6808a68298be8b1bc7248b618ca12a0434fa824bb6efb53c7d2
.pes/objects/d2/6c424b937840baefb0d18b98a75658c2848fa5cb7b088e33e1ae52ba8c1921
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f7/d9e9acbbee29ce9dabb78204cc915faba08922a014b1dbc6f63f8119eb1350
.pes/objects/fc/e3294e2c12d63a4b971d0aed5179c4ad9d28bbae27f6367a7740e51db1a071
.pes/refs/heads/main
```

4 C:

```
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ cat .pes/refs/heads/main
fce3294e2c12d63a4b971d0aed5179c4ad9d28bbae27f6367a7740e51db1a071
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final:

```
ritesh@Ritesh:~/OSORANGEU4/PES2UG24CS404-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 444d8f7a1a4a... Initial commit

Log after first commit:
commit 444d8f7a1a4abd2fea5150798eea08779b743273032a4d1b610abc61eb8ed264
Author: Ritesh <PES2UG24CS404>
Date:   1776757946
```

```
    Initial commit


--- Second Commit ---
Committed: 4e0ccbccf1fc... Update file.txt

--- Third Commit ---
Committed: cd12e9015ba2... Add farewell

--- Full History ---
commit cd12e9015ba29c32f2dcdc6ecdb2a9867d3f7784bba564551a8fb9ad085c45d0
Author: Ritesh <PES2UG24CS404>
Date:   1776757946

    Add farewell

commit 4e0ccbccf1fc03786b77f771a2adba97aeb41e405389f31ae7876ba06cd6de8f
Author: Ritesh <PES2UG24CS404>
Date:   1776757946

    Update file.txt

commit 444d8f7a1a4abd2fea5150798eea08779b743273032a4d1b610abc61eb8ed264
Author: Ritesh <PES2UG24CS404>
Date:   1776757946

    Initial committial commitES2Q


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
cd12e9015ba29c32f2dcdc6ecdb2a9867d3f7784bba564551a8fb9ad085c45d0

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
cd12e9015ba29c32f2dcdc6ecdb2a9867d3f7784bba564551a8fb9ad085c45d0

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/44/4d8f7a1a4abd2fea5150798eea08779b743273032a4d1b610abc61eb8ed264
.pes/objects/4e/0ccbccf1fc03786b77f771a2adba97aeb41e405389f31ae7876ba06cd6de8f
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/cd/12e9015ba29c32f2dcdc6ecdb2a9867d3f7784bba564551a8fb9ad085c45d0
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Phase 5:

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans: **Files Changed:** .pes/HEAD must be updated to point to the new branch (e.g., ref: refs/heads/<branch>). The .pes/index must be overwritten with the tree structure of the target commit. The Files must be physically added, deleted, or modified to match the snapshot of the target branch. The operation must be **atomic** and **safe**. It is complex because you must compare the current working directory, the index, and the target tree simultaneously to ensure no unsaved user changes are accidentally deleted.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans: To detect a conflict using only the index and object store, you perform a three-way hash comparison for every tracked file:

1Compare the file hash in the current commit (**HEAD**) to the hash in the **Index**. If they differ, the file is "dirty" (modified or staged).

2 If the file is dirty, compare the **Index** hash to the hash in the **Target Branch** tree.

3 Refuse if**:** (Hash_HEAD != Hash_Index) AND (Hash_Index != Hash_Target).

- *Logic:* If you've changed a file and the branch you are switching to wants that file to look different than your version, checkout must stop to prevent data loss.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans: Commits function normally, with each new commit pointing to the previous one as its parent. However, since no branch file (like main) is being updated, these commits have no "name" to keep them anchored. If you checkout a different branch, the hashes of the commits made while detached are no longer easily reachable. A user can recover by checking the history of where HEAD has been to find the lost commit hash, then creating a new branch at that hash: pes branch recovery <hash>.

Phase 6:

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans: A **Hash Set** is best for lookups. To save memory, a Bloom Filter can act as a fast probabilistic pre-filter, followed by a bit-array if hashes can be mapped to integers. You must visit all 100,000 commits. Assuming an average of 5 unique objects (trees/blobs) per commit due to structural sharing, you would visit approximately 500,000 **to** 700,000 objects. You stop redundant traversal when you hit a hash already in your "marked" set.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans: GC may decide an object is "unreachable" while a concurrent commit process is in the middle of creating it but hasn't updated a branch reference to point to it yet.

Race Condition Scenario:

1. **Commit Process:** Creates a new blob object and calculates its hash.
2. **GC Process:** Scans refs, doesn't see the new hash yet (as the commit hasn't finished), and marks the blob for deletion.
3. **Commit Process:** Finally writes the commit object pointing to that blob.
4. **GC Process:** Deletes the blob. The new commit is now broken/corrupt.

Git uses a "prune expiration" threshold (defaulting to 14 days). It checks the mtime (last modification time) of every object file; if an object is unreachable but was created within the grace period, GC refuses to delete it, assuming it belongs to an active, uncompleted operation.